

Solution Requirements

Date	30 September 2026
Team ID	6abc94497a3f72d0b9a67d15
Project Name	PocketSmart AI: Your Smart Budget and Recommendation Assistant
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Authentication	Users can log in/verify identity through the project authentication flow implemented in auth.py.	High
1	Authorization levels	The solution is designed for authenticated users; no separate multi-role permission model is specified.	Medium
2	External interfaces	Connects to Google Gemini through the google-genai SDK for AI-generated recommendations.	High
3			

		<i>services the system must connect to]</i>	
4	Transactions processing	Handles planner requests and exchanges user inputs with the FastAPI backend and AI recommendation service; no payment transaction is specified.	High
5	Reporting	Displays generated recommendations/results to the user for the selected planning scenario.	Medium
6	Business rules, etc.	Recommendations should respect the user's stated budget and planning context such as home interior, party or jewelry requirements.	High
7	Compliance to laws or regulations	No specific legal or industry standard is identified in the project information; user data and credentials should be handled responsibly.	Medium
8	Other	Provide three core planner flows: Home Interior Planner, Party Planner and Jewelry Planner.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria

1	Performance & Speed	Planner pages and normal form interactions should respond smoothly; AI response time depends on the external Gemini service.	Responsive during normal use; no numeric response-time target is specified.
2	Scalability	Use modular FastAPI routes, services and models so additional planner features can be added without redesigning the whole application.	New modules can be added without changing the core architecture.
3	Security & Data Privacy	Protect authentication data and avoid exposing sensitive credentials such as API keys in client-side code.	Authentication flow works and secrets remain server-side.
4	Reliability & Availability	Handle invalid inputs and external AI-service failures without crashing the user-facing application.	Graceful error handling for normal failure cases.
5	Usability & Accessibility	Provide clear HTML-based planner forms and readable recommendation results across the core modules.	Users can complete a planner flow and understand the returned recommendations.
6	Other	Maintainable, modular Python/FastAPI code with reusable templates and static assets.	Code remains organized across routes, models, utilities, templates and static files.